

Final Project: Individual Report

Nitish Devrani (Team Stream Dreamers)

October 9, 2026

1 Introduction

The Stream Dreamers team developed a real-time data monitoring dashboard using Next.js to process and visualize streaming data from Kafka via WebSockets.

The dashboard provides critical insights such as:

- Live query execution statistics (query count, execution time, scanned and spilled data).
- Real-time alerts based on threshold violations.
- Predictive analytics for future query performance.
- User-specific billing information based on resource consumption.

The system is optimized to handle high-frequency real-time updates while maintaining performance and scalability.

2 System Architecture

The architecture consists of several components working together to process raw data, generate insights, and display it on a real-time dashboard.

- **Data Storage:** Raw data is stored in DuckDB.
- **Kafka Producer:** Streams processed query execution data.
- **Kafka Consumer:** Retrieves streamed data and sends it for further processing.
- **Data Processing:** Cleans, aggregates, and enriches data before sending it to Kafka.
- **WebSockets:** Delivers real-time data to the Next.js dashboard.
- **Next.js Frontend:** Displays real-time visualizations using Recharts.
- **Metrics and Prediction:** Uses machine learning to predict query behavior.

3 My Role in the Project

My primary responsibility was to design and implement the real-time dashboard using Next.js.

Key contributions:

- Built a dynamic, real-time **Next.js** dashboard with WebSockets.
- Integrated **Recharts** for visual analytics, including:
 - Line charts for query execution trends.
 - Stacked bar charts for query type distributions.
 - Billing summaries for user costs.
- Implemented **WebSockets** to enable real-time updates without page refreshes.
- Optimized data flow to show only the **last 10 data points** for performance.
- Developed an **alerting system** that:
 - Displays threshold alerts dynamically.
 - Automatically removes old alerts after 5 seconds.
 - Provides a dropdown for viewing historical alerts.
- Managed user billing visualization by extracting per-user execution costs.

4 Why Choose Next.js?

4.1 Comparison with Other Dashboard Tools

Next.js was chosen because of:

- Superior real-time data handling using WebSockets.
- Full customization with React components.
- Performance optimization through Server-Side Rendering (SSR).
- No licensing costs unlike PowerBI.

Feature	Next.js	Streamlit	PowerBI
Real-Time Updates	Yes	Limited	No
Custom UI/UX	Fully Customizable	Limited Components	Predefined UI
Scalability	High	Limited	Medium
Cost	Free	Free	Paid Subscription
Web Integration	Yes (API, WebSockets)	No	Limited

Table 1: Comparison of Next.js, Streamlit, and PowerBI

5 Dashboard Features

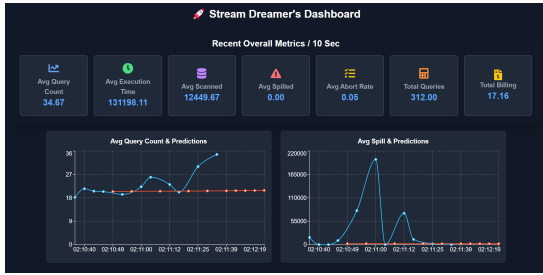


Figure 1: Next.js Real-Time Overall Data

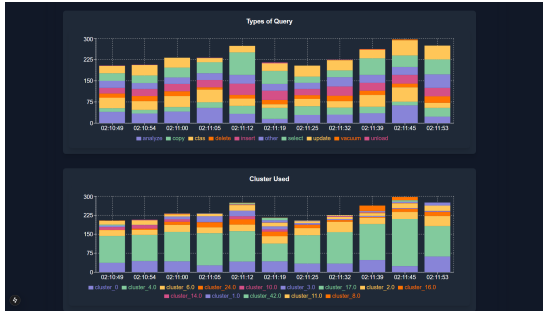


Figure 2: Next.js Real-Time User Based Data

5.1 Line Charts

- Query Execution Trends: Tracks query count over time.
- Predictions: Compares real-time vs predicted query count.

5.2 Stacked Bar Charts

- Query Type Breakdown: Visualizes frequency of SELECT, UPDATE, etc.
- Cluster Utilization: Tracks queries per cluster.

5.3 User Billing Summary

- Displays real-time cost tracking per user.
- Includes serverless usage metrics.

6 Challenges and Solutions

6.1 Handling Large Data Streams

Problem: Kafka streams send data every few seconds, leading to overload. Solution: Implemented data slicing, showing only the last 10 entries.

6.2 Synchronizing Real and Predicted Data

Problem: Predictions arrive in batches, real data is continuous. Solution: Used a buffering mechanism to append predictions every 10 updates.

6.3 WebSocket Connection Drops

Problem: WebSockets disconnected due to high-frequency messages. Solution: Implemented auto-reconnect logic in Next.js.

7 Conclusion and Future Work

This real-time Kafka dashboard successfully visualizes query execution metrics, user billing, and ML-based predictions.

Future Enhancements:

- Improved ML models for more accurate predictions.
- Dark/Light Theme Toggle for UI customization.
- Multi-Tenant Support for multiple Kafka topics.

References

1. Next.js Official Docs - <https://nextjs.org/docs>
2. WebSockets in React - <https://developer.mozilla.org/en-US/docs/Web/API/WebSockets>